

First Half of 2025 Chonnam National University PIMM Algorithm Party

Solutions Commentary Editorial

PIMM Algorithm Study Group, Chonnam National University Game Development
Club

This Contest is operated by

Ko Min-
gyu^{jjkmk1013}

Kim Geun-
seong^{onsbtyd}

Park
Jonghyeon^{belline0124}

Lee Yun-
su^{lys9546}

Jeong
Yeong-
do^{0do}

Choi
Jeong-
hwan^{jh01533}

dongwook7

lycoris1600

realpsdoingdamyoo

sjhi00

tony9402

utilforever

with .

Prize Draw

- Participants confirmed to have cheated were excluded.

For details, see the AI Use Assessment Report on Contest Source Code Submissions published alongside this document in the [editorial repository](#).

Burger King Whopper Set Five winners drawn with weights equal to the cube of the number of problems solved

- kwoncycle • dk10211 • ckj31110 • golazcc83 • fermion5

Iced Caffè Americano (Tall) Five winners drawn with weights equal to the square of the number of problems solved

- luciaholic • oh040411 • nemomaru • ssjjss • asker5325

Iced Caffè Americano (Tall) Five winners drawn with weights equal to the number of problems solved

- patata22
- minpro0818
- starboard
- ssafyiwan
- eka

Iced Caffè Americano (Tall) Five winners drawn

- tph01198
- didtldms2525
- aru0504
- nabina1395
- tkvl94

Participation Commemorative solved.ac Profile Background / Badge

- Participants confirmed to have cheated were excluded.

For details, see the AI Use Assessment Report on Contest Source Code Submissions published alongside this document in the [editorial repository](#).



Profile background: Outside the Treasure Chest

Club banner background for the 2025 spring commencement ceremony

© Jeong Hui-su

Profile badge: A Slice of Cake

Amangchu Team, an asset from the game University Café Tycoon

© An Sang-i

Problem		Difficulty	Author
A	Dementia Prevention Rules 3.3.3	Easy	Lee Yun-su ^{lys9546}
B	Secret Strike (Small)	Normal	Lee Yun-su ^{lys9546}
C	My Name is Tree	Hard	Kim Geun-seong ^{onsbtyd}
D	Rolling Blocks	Hard	Ko Min-gyu ^{jjkmk1013}
E	Stacking Blocks	Hard	Jeong Yeong-do ^{0do}
F	Min Max Mex	Hard	Choi Jeong-hwan ^{jh01533}
G	Treasure Hunt	Challenging	Choi Jeong-hwan ^{jh01533}
H	Secret Strike (Large)	Challenging	Lee Yun-su ^{lys9546} , Choi Jeong-hwan ^{jh01533}

A. Dementia Prevention Rules 3.3.3

#string, #parsing

Difficulty — **Easy**

- Author: Lee Yun-su^{lys9546}
- 237 Submitted, 186 Passed
AC Ratio: 79.747%
- First Solver: 14cheung, +1min

A. Dementia Prevention Rules 3.3.3

- ✓ Implement exactly what the problem asks.
- ✓ Split the given string on ., |, :, and # to extract the numbers, then add them all together.

B. Secret Strike (Small)

#implementation, #bruteforcing

Difficulty — **Normal**

- Author: Lee Yun-su^{lys9546}
- 326 Submitted, 102 Passed
AC Ratio: 33.436%
- First Solver: 14cheung, +5min

B. Secret Strike (Small)

- ✓ Since $S \leq 100$ and $K = 2$, trying a Secret Strike on every pair fits within the time limit.
- ✓ Trying all pairs takes $O(N^2)$ time, so trying it twice ($K = 2$) takes $O(N^4)$ time.

C. My Name is Tree

#graph, #bfs, #multisource_bfs, #shortest_path

Difficulty — **Hard**

- Author: Kim Geun-seong^{onsbtyd}
- 142 Submitted, 26 Passed
AC Ratio: 19.014%
- First Solver: sadtreap, +8min

C. My Name is Tree

Problem objective

- ✓ Determine whether two nodes with the same name lie within distance K of each other.

Solving with BFS/DFS

- ✓ Run BFS or DFS from every named node to check whether another node with the same name is within distance K .
- ✓ If there are N named nodes, the $O(N^2)$ time complexity causes a time-limit exceeded verdict.
- ✓ Searches for the same name overlap, so we need to reduce the number of searches.

C. My Name is Tree

Solving with multi-source BFS

- ✓ Searching from all nodes with a given name at once reduces the number of searches.
 - ✓ There are at most 1,000 distinct possible names, so we can run multi-source BFS for each name.
1. When exploring neighbors of an arbitrary node A , find a previously visited node B that has a different starting point.
Then distance between the closest pair with the same name = distance recorded at A + distance recorded at B + 1.
 2. If the distance between the closest pair with the same name is at most K , print POWERFUL CODING JungHwan; otherwise, print so sad.

C. My Name is Tree

- ✓ The time complexity is $O(\text{number of distinct names} \times N)$.
- ✓ Jeong-hwan's original name was "Choi Max-Value-Finding Max Function."

D. Rolling Blocks

#implementation, #dp

Difficulty — **Hard**

- Author: Ko Min-gyu^{jjkmk1013}
- 57 Submitted, 16 Passed
AC Ratio: 28.07%
- First Solver: 14cheung, +27min

D. Rolling Blocks

Problem objective

- ✓ Count the starting positions from which the block can reach the designated goal tile in exactly K moves.

Solution idea

- ✓ Block moves are reversible, so we can exchange the starting and ending positions.
- ✓ Equivalently, start at the goal tile and count all ordinary tiles on which the block can end upright after exactly K moves.

D. Rolling Blocks

Solving with DP

- ✓ $DP_{i,j,k,d}$ = whether the block can reach tile (i, j) in state d after k moves.
- ✓ Represent the block's state using three or five types.
 - One: the block is upright.
 - Two or four: the block is lying down (the number depends on its orientation).

D. Rolling Blocks

- ✓ The time complexity is $O(NMK)$.
- ✓ The constraints prohibit placing the block on the goal tile.
If the block can return to the goal tile after K moves, exclude that case.

E. Stacking Blocks

#implementation

Difficulty — **Hard**

- Author: Jeong Yeong-do^{0do}
- 44 Submitted, 16 Passed
AC Ratio: 36.364%
- First Solver: sorohue, +67min

E. Stacking Blocks

When $M = 1$

- ✓ $N = 200,000, Q = 500,000$.
- ✓ Process each query in at most $O(\log N)$ time.
- ✓ A FRONT query outputs the number of STACK queries, so it takes $O(1)$ time.
- ✓ For $M = 1$, a SIDE query asks for the maximum height of all block stacks. Use binary search or a segment tree to store and retrieve it in $O(\log N)$ time.
- ✓ Process a TOP query in $O(1)$ time using a map.
- ✓ A STACK or REMOVE query changes a block stack's height by 1, so manage heights with a map. When the maximum height changes, inspect neighboring stacks' heights to store and retrieve the data in $O(1)$ time.

E. Stacking Blocks

When $M = 200,000$

- ✓ A FRONT query asks for the sum of the maximum heights visible at each coordinate from the front.
 - ✓ Maps support $O(1)$ storage and lookup, so create N maps of heights to process these queries in $O(N + Q)$ time.
- ✓ Process SIDE queries using the same strategy as FRONT in $O(M + Q)$ time.
- ✓ Process TOP queries in $O(1)$ time using a map.

E. Stacking Blocks

- ✓ The solution for the $\langle M = 1 \text{ case} \rangle$ has time complexity $O(Q)$.
- ✓ The solution for the $\langle M = 200,000 \text{ case} \rangle$ has time complexity $O(N + M + Q)$.
With additional optimizations such as binary search, a time complexity of $O(\log NQ + \log MQ)$ may be possible.

F. Min Max Mex

#greedy

Difficulty — **Hard**

- Author: Choi Jeong-hwan^{jh01533}
- 136 Submitted, 33 Passed
AC Ratio: 24.265%
- First Solver: sorohue, +7min

F. Min Max Mex

Problem objective

- ✓ Find the smallest nonnegative integer that can be made absent by removing K elements.

Min Mex

- ✓ Count the occurrences of each array element.
- ✓ Starting from 0, check whether each element occurs at most K times.

Max Mex

1. Sort the array.
2. Append a large number INF outside the input bounds to the end of the array.
3. Traverse the array, greedily filling any gaps between consecutive values.

G. Treasure Hunt

#math, #constructive, #interactive

Difficulty — **Challenging**

- Author: Choi Jeong-hwan^{jh01533}
- 50 Submitted, 5 Passed
AC Ratio: 10%
- First Solver: fermion5, +276min

G. Treasure Hunt

A method without finding X

- ✓ The notes at two positions with equal Manhattan distance remain equal even after each is XORed with X .
- ✓ The notes at two positions with different Manhattan distances remain different even after each is XORed with X .
- ✓ If N is large, binary search or a similar technique can solve the problem on a logarithmic scale.

G. Treasure Hunt

Examples of the method without finding X

1. Digging at the following positions leaves two candidates.

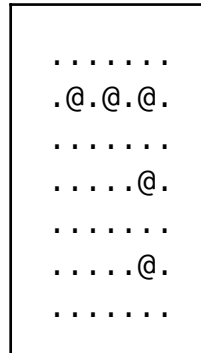
```
.....  
.@....  
..@...  
...@..  
....@.  
.....
```

2. When N is even, one candidate remains.

```
@.@.@.  
.....  
.....  
.....@  
.....  
.....@
```

G. Treasure Hunt

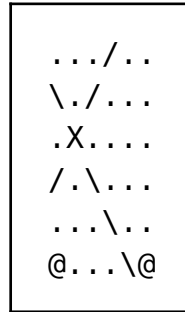
3. When N is odd, two candidates remain.



G. Treasure Hunt

A method that finds X

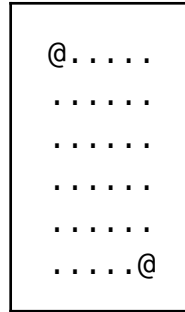
✓ If X is known, the problem can be solved with three queries.



```
.../..
\./...
.X....
/.\...
...\.
@... \@
```

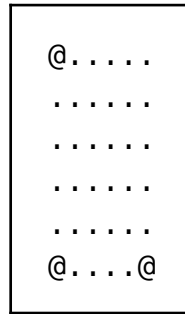
✓ One candidate remains.

G. Treasure Hunt



- ✓ Let the responses to these queries be A and B , and let the responses without applying X be a and b . Then:
 - ✓ $a + b = 2N - 2$
 - ✓ $(A \oplus X) + (B \oplus X) = 2N - 2$
- ✓ The number of possible values of X is less than $N/2$.

G. Treasure Hunt



- ✓ After asking these queries, dig at all candidate positions for X to solve the problem.

G. Treasure Hunt

Further thought

✓ Can we determine X exactly using only a constant number P of queries?

H. Secret Strike (Large)

#dp

Difficulty — **Challenging**

- Authors: Lee Yun-su^{lys9546}, Choi Jeong-hwan^{jh01533}
- 7 Submitted, 28 Passed
AC Ratio: 25%
- First Solver: sorohue, +60min

H. Secret Strike (Large)

- ✓ Property 1. If we call the two characters struck by a Secret Strike and the substring between them an interval, there is an optimal solution in which intervals neither overlap nor contain one another.
- ✓ Property 2. There is always an optimal solution that applies at most one Secret Strike to each letter of the alphabet.

H. Secret Strike (Large)

<Property 1. There is an optimal solution in which intervals neither overlap nor contain one another.>

- ✓ If an interval A contains another interval B , omitting the Secret Strike on B gives the same result. Thus there is an optimal solution without containment.

H. Secret Strike (Large)

- ✓ $DP_{i,j} = \max(\text{length of intervals removable after examining through position } i \text{ with up to } j \text{ Secret Strikes})$
- ✓ Transitions
 - ✓ When applying a Secret Strike involving the i -th character, consider all k such that $S_{i-k} = S_i$, taking $\max(DP_{i-k,j-1})$.
 - ✓ When not applying a Secret Strike involving the i -th character, take $DP_{i-1,j}$.
- ✓ This computation takes $O(N^2K)$ time, but another DP optimization reduces it to $O(NK)$.
- ✓ $DP'_{c,j} = \max(\text{over all indices } i \text{ where } S_i = c, \text{ the maximum number of characters removable with } j \text{ Secret Strikes through } i - 1, \text{ plus } n - i)$

H. Secret Strike (Large)

<Property 2. There is always an optimal solution that applies at most one Secret Strike to each letter.>

..[a....a]....[a....a]..

Example: Striking the two outermost as is better than striking each [a...a] separately.

- ✓ Rather than applying two Secret Strikes to four occurrences of the same letter, it is more optimal to strike the two outermost occurrences once.
- ✓ The time complexity is $O(NL)$ ($L = \text{number of letters other than } X \leq 25$).